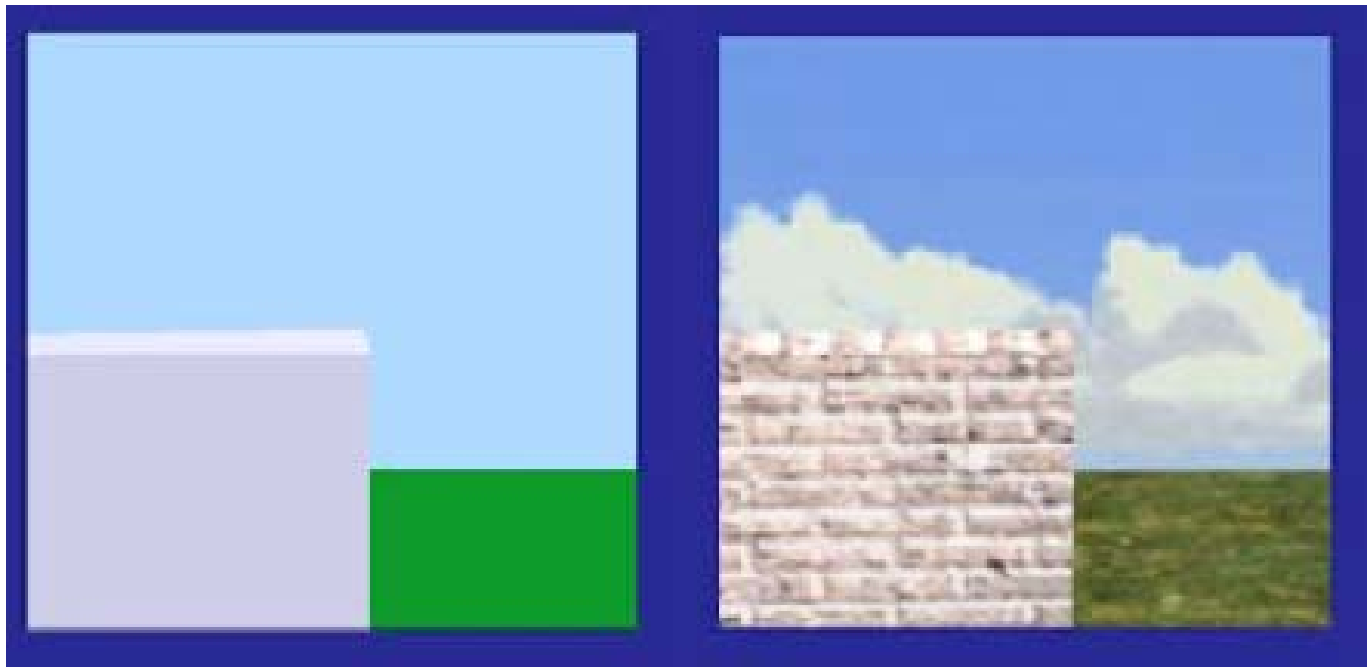
OpenGL三维图形编程

纹理映射（上）

北京大学计算机系人机交互与多媒体实验室

概述

- 通俗的说
  - 纹理（texture）就是图片
  - 纹理映射（texture mapping）技术 就是把纹理图片贴到物体表面上，使得表面变成自己所希望的样子（颜色、图案、凹凸感……）



概述

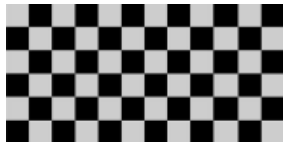
- 物体的表面几何存在三种基本的度量层次
  - 宏观结构层次（macrostructure）
    - 物体的几何模型，一般用多边形、参数曲面描述
  - 亚宏观结构层次（mesostructure）
    - 相当小但仍看得见的几何细节，如砖墙上的凹凸部分
  - 微观结构层次（microstructure）
    - 描述肉眼看不到的表面细节和光学特性

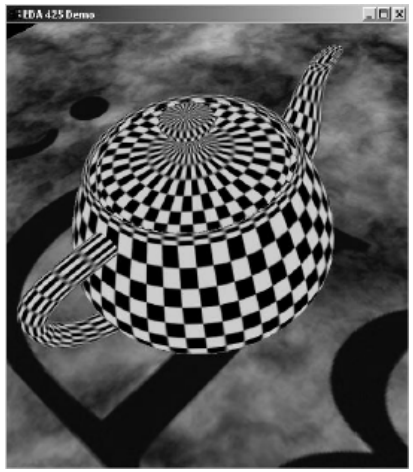
概述

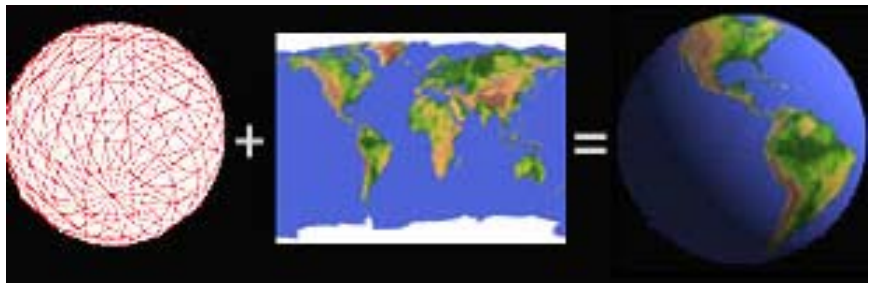
- 纹理映射一般针对前两个几何层次进行
  - 在宏观层次，使用颜色纹理映射可为物体表面贴上真实的色彩花纹
  - 在亚宏观层次，则采用更为精细的纹理映射技术表现凹凸不平的表面纹理细节，如凹凸纹理映射，BTF（双向纹理函数）等等

概述

- 我们只考虑宏观层次的纹理映射



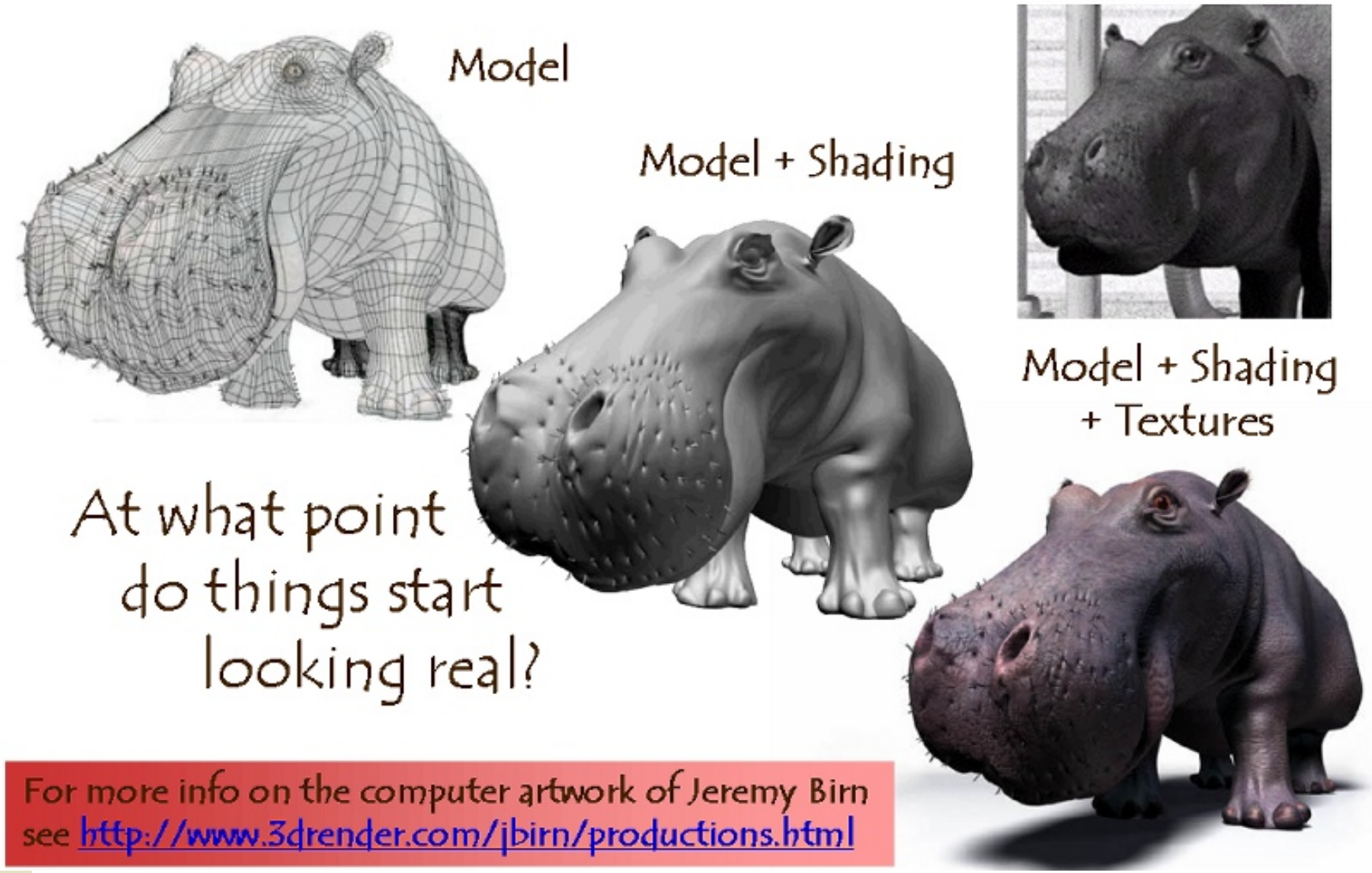




概述

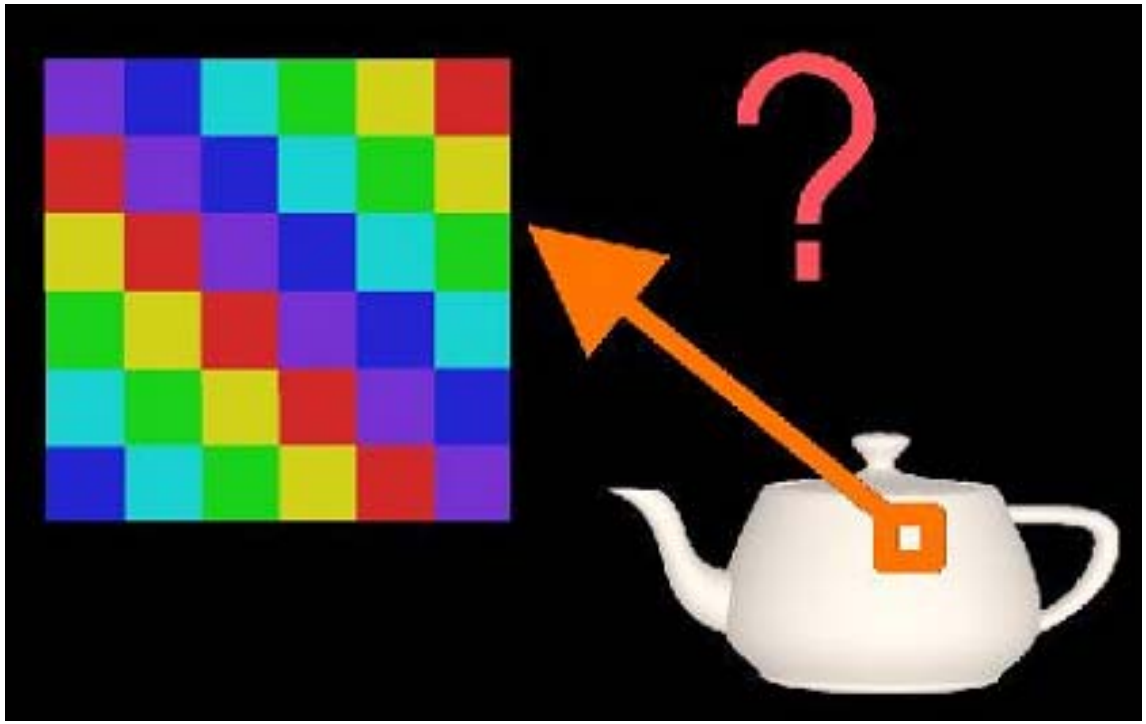
- 为什么要进行纹理映射
  - 仅有光照的场景看上去并不很真实
  - 模型的多边形可能有些粗糙、不平滑
  - 纹理可以对很多外观效果因素进行调制,如
    - 反射效果，表面颜色，镜面高光，透明度，表面细节……
  - 用户定制物体表面细节

概述


Model
Model + Shading
Model + Shading
+ Textures
At what point
do things start
looking real?
For more info on the computer artwork of Jeremy Birn
see http://www.3drender.com/jbirn/productions.html

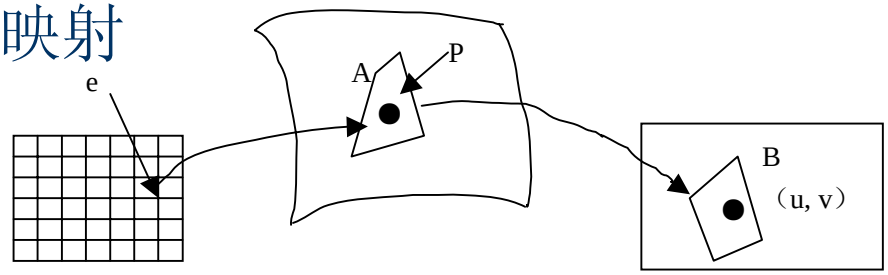
概述

- 如何找到纹理与物体表面之间的对应关系？



概述

- 两个空间
  - 一般用一幅数字化图像来离散定义颜色纹理，该平面区域称作纹理空间
  - 被映射的物体表面一般是参数曲面或多边形网格，称为景物空间
- 纹理映射的实质是建立两个映射关系：
  - 从屏幕空间到纹理空间的映射
  - 从纹理空间到景物空间的映射



$t(e) = P$ $m(P) = (u, v)$

基本流程

- 创建纹理对象
- 指定纹理内容
- 指定各种纹理属性
- 提供纹理坐标
- 激活纹理功能

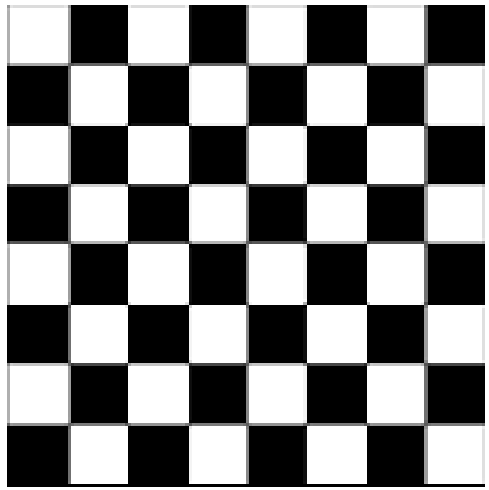
基本流程

- 一个简单的例子：checker\Debug\checker.exe

基本流程

- 棋盘纹理生成分析

```
void makeCheckImage(void)
{
  int i, j, c;
  for (i = 0; i < checkImageHeight; i++) {
    for (j = 0; j < checkImageWidth; j++) {
      c = ((((i&0x8)==0)^(j&0x8)==0))*255;
      checkImage[i][j][0] = (GLubyte) c;
      checkImage[i][j][1] = (GLubyte) c;
      checkImage[i][j][2] = (GLubyte) c;
      checkImage[i][j][3] = (GLubyte) 255;
    }
  }
}
```



纹理对象

- 命名纹理对象
- 创建、使用纹理对象
- 清除纹理对象

纹理对象

- 命名纹理对象
  - 每个纹理都有一个唯一的名称ID
    - *void* ***glGenTextures****(GLsizei n, GLuint *textureNames)*
  - 确定纹理名称是否正在使用
    - *GLboolean* ***glIsTexture****(GLuint textureName)*
  - 0是受保护的纹理名称

纹理对象

- 创建、使用纹理对象
  - *void* ***glBindTexture****(GLenum target, GLuint textureName);*
  - *target*确定纹理维数
    - *GL_TEXTURE_1D，GL_TEXTURE_2D，GL_TEXTURE_3D*
  - *textureName*指定要绑定的纹理名称

纹理对象

- *glBindTexture*三个功能
  - 以*textureName*为参数第一次调用时，创建一个新的纹理对象

```
void init(void)
{
    ……
    glGenTextures(1, &texName);
    glBindTexture(GL_TEXTURE_2D, texName);
    ……
}
```

纹理对象

- *glBindTexture*三个功能
  - 再次以*textureName*为参数调用时，使该对象成为活动状态

```
void display(void)
{
    ……
    glBindTexture(GL_TEXTURE_2D, texName);
    glBegin(GL_QUADS);
    ……
}
```

纹理对象

- *glBindTexture*三个功能
  - 以*textureName*为0调用时，OpenGL停止使用纹理对象

纹理对象

- 清除纹理对象
  - 一旦创建，不论激活与否，纹理对象都处于纹理资源中
  - 释放纹理资源
    - *void* ***glDeleteTextures****(GLsizei n, const GLuint *textureNames)*

指定二维纹理

- 从内存纹理数据创建纹理
  - 先将纹理文件读入内存，然后创建纹理
  - *void* ***glTexImage2D****(GLenum target, GLint level, GLint internalFormat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)*
    - *target* — *GL_TEXTURE_2D 或者 GL_PROXY_TEXTURE_2D*（纹理代理）
    - *level* — 采用多分辨率纹理时，*level*设为相应值，单分辨率情况下为0
    - *internalFormat* — 纹理的内部格式，从38个常量中选择

指定二维纹理

- 从内存纹理数据创建纹理
  - *void* ***glTexImage2D****(GLenum target, GLint level, GLint internalFormat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)*
    - *width*，*height*，*border* — 纹理图像数据的尺寸与边界，*width*、*height*必须为2的幂次，即2^m的形式
    - *format*，*type* — 纹理图像数据的格式与类型，与函数*glReadPixels()*对应参数相同
    - *pixels* — 纹理图像数据

指定二维纹理

- 从内存纹理数据创建纹理
  - *void **glTexImage2D**(GLenum target, GLint level, GLint internalFormat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)*

```
void init(void)
{
    ……
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA,
    checkImageWidth, checkImageHeight, 0, GL_RGBA,
    GL_UNSIGNED_BYTE, checkImage);
    ……
}
```

指定二维纹理

- 调整纹理尺寸
  - 改变原始纹理图像的宽、高，使之满足2的幂次
  - *int* ***gluScaleImage****(GLenum format, GLint widthin, GLint heightin, GLenum typein, const void *datain, GLint widthout, GLint heightout, GLenum typeout, void *dataout)*
    - *format*，*typein*，*typeout* — 函数*glReadPixels()*所支持的任一种格式和类型
    - *widthin*，*heightin*，*datain*，*widthout*，*heightout*，*dataout* — 输入输出的格式和文件内容
    - 必须预先为输出缓存分配空间

指定二维纹理

- 从帧缓存数据创建纹理
  - 帧缓存也可以作为纹理数据的来源
  - *void* ***glCopyTexImage2D****(GLenum target, GLint level, GLint internalFormat, GLint x, GLint y, GLsizei width, GLsizei height, GLint border)*
    - *target* — 只能是*GL_TEXTURE_2D*
    - *x*，*y* — 从帧缓存的左下角位置（*x*，*y*）开始获取纹理数据
    - 其余参数同*glTexImage2D()*函数

指定二维纹理

- 使用内存数据替换纹理图像
  - 创建纹理的费用远高于修改现有纹理的费用
  - 修改当前纹理图像

指定二维纹理

- 使用内存数据替换纹理图像
  - *void **glTexSubImage2D**(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLsizei width, GLsizei height, GLenum format, GLenum type, const GLvoid *pixels)*
    - *target* — 只能是*GL_TEXTURE_2D*
    - *level* — 在哪个细节层次进行替换
    - *xoffset*，*yoffset* — 替换起始位置，纹理左下角是（*0*，*0*）
    - *width*，*height* — 替换区域的大小，没有2的幂次的限制
    - *format*，*type* — 同*glTexImage2D()*函数
    - *pixels* — 纹理数据

指定二维纹理

- 使用内存数据替换纹理图像
  - 程序示例：texsub\Debug\texsub.exe

```
void keyboard (unsigned char key, int x, int y)
{
    ……
    glBindTexture(GL_TEXTURE_2D, texName);
    glTexSubImage2D(GL_TEXTURE_2D, 0, 12, 44,
    subImageWidth, subImageHeight, GL_RGBA,
    GL_UNSIGNED_BYTE, subImage);
    ……
}
```

指定二维纹理

- 使用帧缓存数据替换纹理图像
  - 强调：帧缓存可以作为纹理数据的来源
  - *void* ***glCopyTexSubImage2D****(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLint x, GLint y, GLsizei width, GLsizei height)*
    - 参数意义与*glCopyTexImage2D()*、*glTexSubImage2D()*类似

纹理 vs. 表面颜色

- 物体表面颜色
  - 通过光照，物体表面已经指定颜色；没有光照时，也可以直接为物体表面指定颜色
  - 纹理也为物体表面附上颜色
- 两种颜色如何结合？如何确定最终表面颜色？

纹理 vs. 表面颜色

- 两种颜色结合方式
  - 四种结合方式（在原始颜色上如何应用纹理）
    - 替换（*GL_REPLACE*）
    - 调节（*GL_MODULATE*）
    - 贴花（*GL_DECAL*）
    - 混合（*GL_BLEND*）

纹理 vs. 表面颜色

- 功能函数
  - *void* ***glTexEnv****{if}(GLenum target, GLenum pname, TYPE param)*
    - *target* — 必须为*GL_TEXTURE_ENV*
    - *pname* — 必须为*GL_TEXTURE_ENV_MODE*
    - *param* — 四种结合方式之一：*GL_REPLACE*，*GL_MODULATE*，*GL_DECAL*，*GL_BLEND*

纹理 vs. 表面颜色

- 四种结合方式，和纹理内部格式一起，决定了物体表面的最终颜色
  - 内部格式，即*glTexImage2D()*的第三个参数*internalFormat*，从38个内部格式常量中取值
  - 38个内部格式常量，分为六种基准内部格式
    - GL_ALPHA (A)
    - GL_LUMINANCE (L)
    - GL_LUMINANCE_ALPHA (L and A)
    - GL_INTENSITY (I)
    - GL_RGB (C)
    - GL_RGBA (C and A)

纹理 vs. 表面颜色

- 混合方式
  - 还需要额外指定一个混合颜色
  - *void* ***glTexEnv****{if}****v****(GLenum target, GLenum pname, TYPE *param)*
    - *target* — 必须为*GL_TEXTURE_ENV*
    - *pname* — 必须为*GL_TEXTURE_ENV_COLOR*
    - *param* — 用于混合的R，G，B，A值，四元数组

纹理 vs. 表面颜色

- 替换、调节功能和基准内部格式结合

Base Internal Format	Replace Texture Function	Modulate Texture Function
GL_ALPHA	C = Cf, A = At	C = Cf, A = AfAt
GL_LUMINANCE	C = Lt, A = Af	C = CfLt, A = Af
GL_LUMINANCE_ALPHA	C = Lt, A = At	C = CfLt, A = AfAt
GL_INTENSITY	C = It, A = It	C = CfIt, A = AfIt
GL_RGB	C = Ct, A = Af	C = CfCt, A = Af
GL_RGBA	C = Ct, A = At	C = CfCt, A = AfAt

纹理 vs. 表面颜色

- 贴花、混合功能和基准内部格式结合

Base Internal Format	Decal Texture Function	Blend Texture Function
GL_ALPHA	undefined	C = Cf, A = AfAt
GL_LUMINANCE	undefined	C = Cf(1-Lt) + CcLt, A = Af
GL_LUMINANCE_ALPHA	undefined	C = Cf(1-Lt) + CcLt, A = AfAt
GL_INTENSITY	undefined	C = Cf(1-It) + CcIt, A = Af(1-It) + AcIt,
GL_RGB	C = Ct, A = Af	C = Cf(1-Ct) + CcCt, A = Af
GL_RGBA	C = Cf(1-At) + CtAt, A = Af	C = Cf(1-Ct) + CcCt, A = AfAt

纹理 vs. 表面颜色

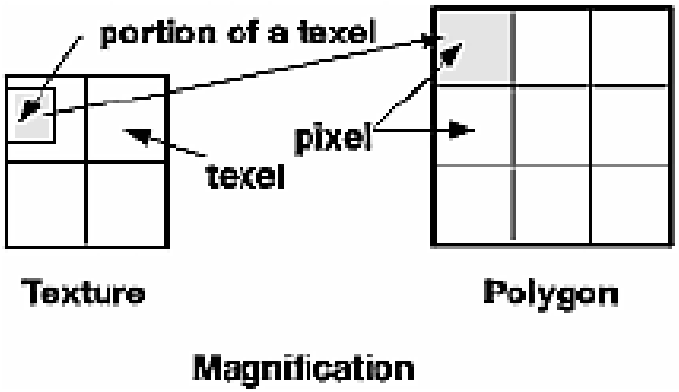
- 镜面颜色与纹理分开使用
  - 光照计算两个颜色
    - 主颜色，所有非镜面属性的和
    - 从颜色，镜面颜色
  - 主颜色和纹理结合，之后加上从颜色
  - 在被照亮的、加纹理的对象上产生高光效果

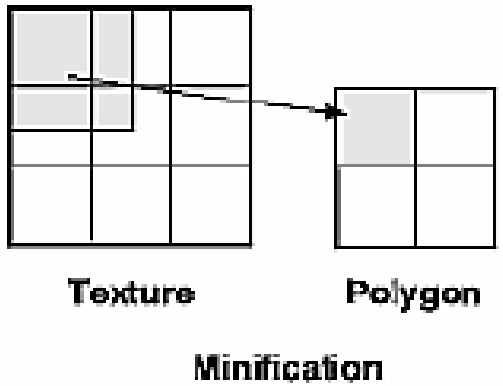
教程

- N.Robins's tutorials， texture.exe

过滤处理

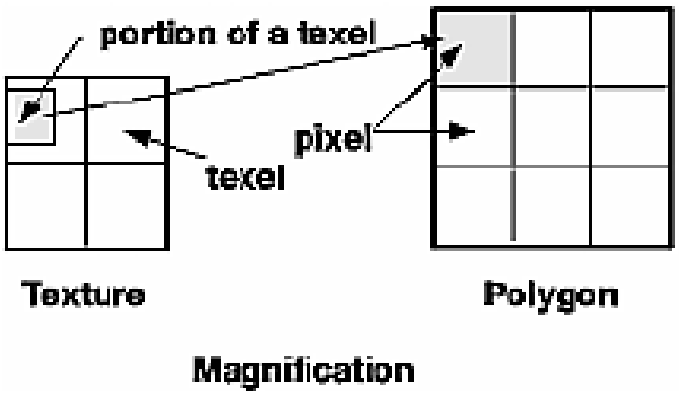
- 纹理图最终映射到屏幕像素
- 纹理图单元（纹元，texel）和屏幕像素单元大小往往不是一对一的匹配

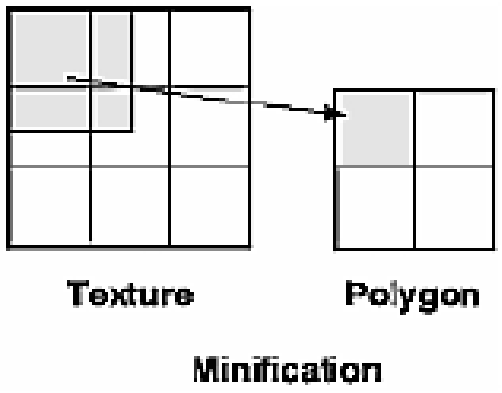




过滤处理

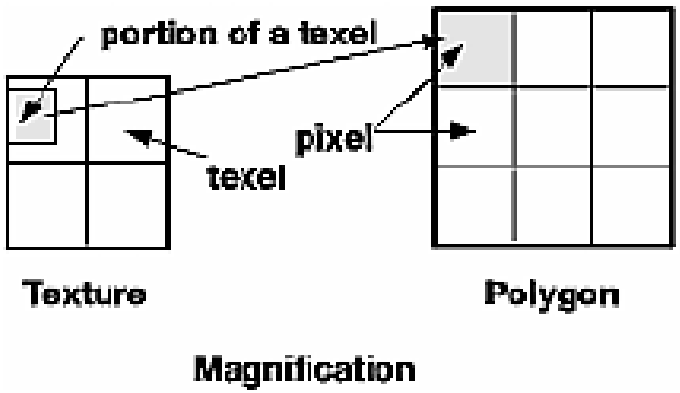
- 必须采取某种计算方式，以确定像素和纹理元素的对应关系

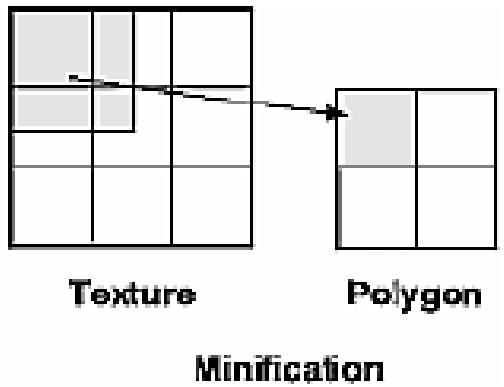




过滤处理

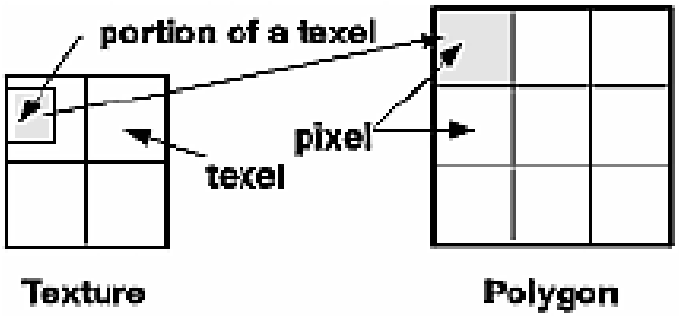
- 最近点匹配（GL_NEAREST）
  - 选择坐标离像素中心最近的纹理单元的颜色作为该像素颜色
  - 计算简单，但可能会导致锯齿现象



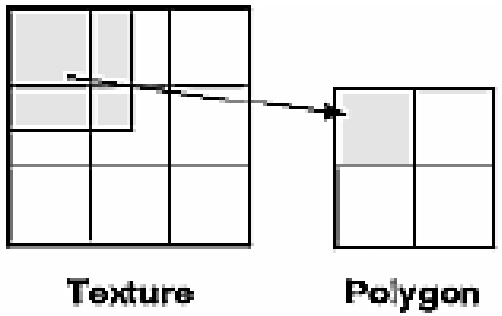


过滤处理

- 线性匹配（GL_NEAREST）
  - 对像素覆盖的所有纹理单元颜色进行插值，计算该像素颜色
  - 计算量偏大，但可以提供更平滑的效果



Magnification



Minification

过滤处理

- 功能函数
  - *void* ***glTexParameter****{if}(GLenum target, GLenum pname, TYPE param)*
    - *target* — 在二维情况下，为GL_TEXTURE_2D
    - *pname* — GL_TEXTURE_MAG_FILTER或GL_TEXTURE_MIN_FILTER，表示纹理放大、缩小两种情况
    - *param* — 过滤方式， GL_NEAREST或GL_NEAREST

教程

- N.Robins's tutorials， texture.exe

纹理坐标

- 每一个顶点应赋予一个纹理坐标
  - 在两个顶点之间的区域，对纹理坐标进行插值，确定该区域特定位置的纹理坐标
- 纹理坐标决定哪一个纹理单元分配给该顶点

纹理坐标

- 依据维数不同，纹理坐标由1～3个坐标组成，（r，s，t）
  - 如果是齐次坐标，则纹理坐标为（r，s，t，q），通常q取1
- 纹理坐标r，s，t的值一般在0～1之间

纹理坐标

- 分配纹理坐标
  - *void* ***glTexCoord****{1234}{sifd}(TYPEcoords)*
  - *void* ***glTexCoord****{1234}{sifd}****v****(TYPE *coords)*

纹理坐标

- 分配纹理坐标
  - 首先调用*glTexCoord*()*分配纹理坐标，然后调用*glVertex*()*将当前纹理坐标分配给顶点

```
void display(void)
{
    ……
    glTexCoord2f(0.0, 0.0);
    glVertex3f(-2.0, -1.0, 0.0);
    ……
}
```

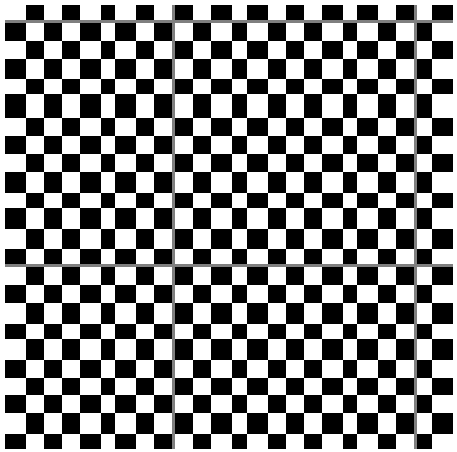
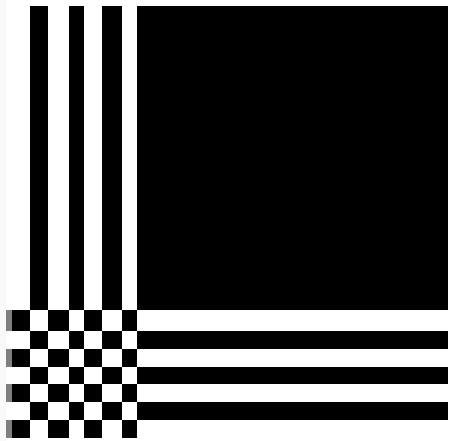
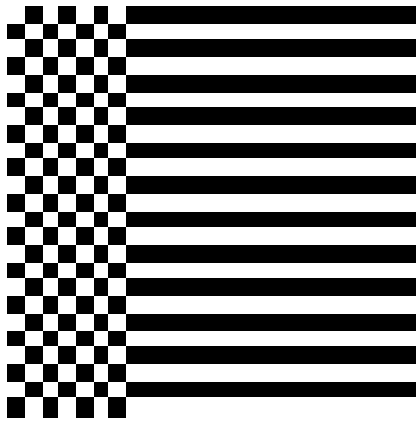
  - 纹理坐标也是状态，OpenGL是状态机

教程

- N.Robins's tutorials， texture.exe

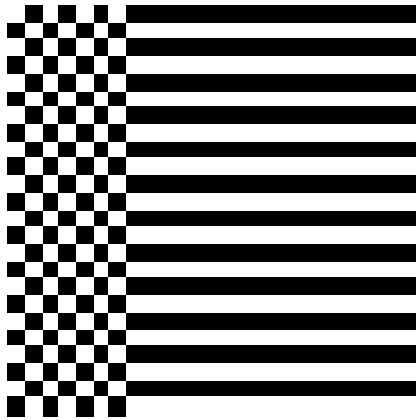
重复和截取纹理

- 当纹理坐标超出[0.0, 1.0]的范围时，在纹理图中对这些坐标进行重复或截取
- 重复或截取可以针对不同的纹理方向进行

重复和截取纹理

- 重复（GL_REPEAT）：忽略纹理坐标的整数部分
- 截取（GL_CLAMP）：将任何大于1.0的值设为1.0，将任何小于0.0的值设为0.0



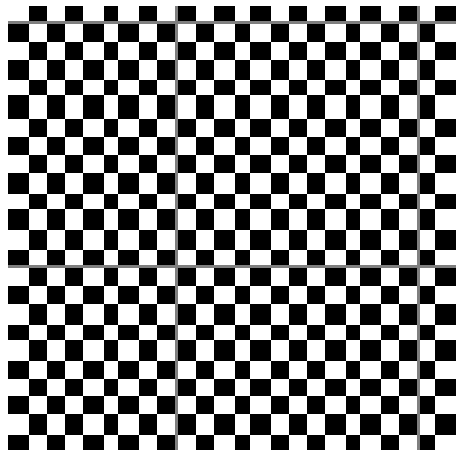
重复和截取纹理

- 功能函数
  - *void* ***glTexParameter****{if}(GLenum target, GLenum pname, TYPE param)*
    - *target* — 在二维情况下，为GL_TEXTURE_2D
    - *pname* — GL_TEXTURE_WRAP_S或GL_TEXTURE_WRAP_T，表示纹理的两个方向
    - *param* — GL_REPEAT或GL_CLAMP，重复或截取

重复和截取纹理

- 重复示例

```
void init(void)
{
    ……
    glTexParameteri(GL_TEXTURE_2D,
    GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D,
    GL_TEXTURE_WRAP_T, GL_REPEAT);
    ……
}
```



重复和截取纹理

- 纹理边界颜色的单独指定
  - *void* ***glTexParameter****{if}****v****(GLenum target, GLenum pname, TYPE *param)*
    - *target* — 在二维情况下，为GL_TEXTURE_2D
    - *pname* — 必须为GL_TEXTURE_BORDER_COLOR
    - *param* — 为边界纹元指定的R，G，B，A值，四元数组

重复和截取纹理

- 重复或截取纹理对线性过滤的影响
  - 线性过滤（GL_LINEAR），实际上是对像素所跨的2×2的纹理单元进行加权平均
  - 在重复纹理（GL_REPEAT）时，2×2的纹理单元忽略边界，如果遇到跨边界的情形，就采用相反边的相应像素
  - 在截取纹理（GL_CLAMP）时，2×2的纹理单元包括边界纹元；如果纹理没有边界，则使用GL_TEXTURE_BORDER_COLOR定义的边界颜色

教程

- N.Robins's tutorials， texture.exe

激活纹理映射

- 使用参数*GL_TEXTURE_2D*调用*glEnable()*函数即可

程序模板

- 初始化（init函数）
  - 命名和创建纹理对象
  - 指定二维纹理
  - 指定各种属性（过滤方式，截取或重复模式，和表面颜色的结合模式）

```
void init(void)
{
  ……
  glGenTextures(1, &texName);
  glBindTexture(GL_TEXTURE_2D, texName);
  ……
  glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
  glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, checkImageWidth, checkImageHeight,
          0, GL_RGBA, GL_UNSIGNED_BYTE, checkImage);
  ……
}
```

程序模板

- 绘制（display函数）
  - 指定一个活动纹理对象
  - 分配纹理坐标，绘制物体

```
void display(void)
{
  ……
  glBindTexture(GL_TEXTURE_2D, texName);
  glBegin(GL_QUADS);
  glTexCoord2f(0.0, 0.0); glVertex3f(-2.0, -1.0, 0.0);
  ……
  glEnd();
  ……
}
```

程序模板

- 所有功能都可以视程序逻辑灵活指定（init函数，display函数，或各种事件响应函数……）
  - 命名、创建纹理对象
  - 指定当前活动纹理对象
  - 替换纹理内容
  - 分配纹理坐标
  - 纹理过滤方式
  - 纹理截取或重复模式
  - 纹理和表面颜色的结合模式
  - 激活纹理映射功能
  - ……
- 它们都只对当前活动的纹理对象产生影响